

Distributed Systems Containerization & Kubernetes Orchestration

MSc DE1 Distributed Systems — Submission Report

Student Name	Harinesh Muthu Kumar
Academic Program	MSc Cloud Computing & Data Engineering
Course	MSc DE1 Distributed Systems
Date	19 September 2026
GitHub Repository	github.com/Hari9489/msc-de1-distributed-systems-docker-k8s
Docker Hub Image	https://hub.docker.com/r/harineshmk/msc-de1-flask-app

Project scope: End to end Flask application containerization, security hardening, Docker image auditing, multi-node Kubernetes orchestration, workload scheduling, service routing and execution verification.

1. Executive Overview & Objectives

This report documents the end-to-end containerization, security hardening, security auditing, and multi-node Kubernetes orchestration of a Python Flask distributed application. The project demonstrates a cloud-native workflow from application packaging through cluster deployment and runtime verification.

Key Objectives

- **Containerization:** Package the Python Flask web application into an optimized, minimal Docker image.
- **Security Hardening:** Enforce non-root execution using UID 10001, restrict privileges, and perform vulnerability analysis with Docker Scout.
- **Multi-Node Orchestration:** Provision a multi-node Kubernetes cluster using Kind and deploy ConfigMap, Deployment, and Service manifests.
- **High Availability & Scheduling:** Maintain multiple replicas across worker nodes with liveness/readiness probes and resource boundaries.
- **Verification & Audit:** Capture runtime evidence, verify endpoint routing, clean temporary binaries, and synchronize repository artifacts.

2. System Architecture & Repository Structure

The repository separates application source code, Kubernetes manifests, Kind cluster configuration, security audit outputs, and execution evidence. This organization provides traceability between implementation files and the evidence used for verification.

```
msc-del-distributed-systems-docker-k8s/
├── app/
│   ├── __init__.py
│   └── routes.py
├── docs/
│   └── Report.pdf
├── evidence/
│   ├── baseline_items.txt
│   ├── baseline_root.txt
│   ├── container_uid.txt
│   ├── container_user.txt
│   ├── healthcheck.txt
│   ├── image_layers.txt
│   ├── k8s_deployment.txt
│   ├── k8s_endpoint_response.txt
│   ├── k8s_pods.txt
│   └── k8s_services.txt
├── k8s/
│   ├── configmap.yaml
│   ├── deployment.yaml
│   └── service.yaml
├── kind/
│   └── cluster-config.yaml
├── security/
│   ├── docker_scout_recommendations.txt
│   ├── docker_scout_vulns.txt
│   └── sbom.json
├── .dockerignore
├── .gitignore
├── compose.yaml
├── Dockerfile
├── LICENSE
├── README.md
├── requirements.txt
└── run.py
```

3. Containerization & Security Engineering

3.1 Dockerfile Design & Optimization

The container uses **python:3.11-slim** to reduce the base image footprint. A dedicated group and user are created with UID/GID 10001. Ownership of the application directory is assigned to that identity and the image explicitly switches away from root before execution. A container health check tests the Flask endpoint.

```
FROM python:3.11-slim
WORKDIR /app

RUN groupadd -g 10001 appgroup && \
    useradd -u 10001 -g appgroup -s /bin/sh -m appuser

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN chown -R appuser:appgroup /app

USER 10001:10001
EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/') || exit 1"

CMD ["python", "run.py"]
```

3.2 Security Auditing & Image Registry

- **Docker Scout:** Vulnerability information and an SBOM are recorded in the security directory.
- **Registry:** The image is documented as published as **harinешmk/msc-de1-flask-app:1.0.0** and **:latest**.
- **Security identity:** Runtime execution uses appuser with UID 10001/GID 10001.

4. Kubernetes Multi-Node Deployment Specification

4.1 Kind Cluster Topology

The cluster configuration defines one control-plane node and two worker nodes, creating a three-node environment for distributed workload scheduling.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: msc-cluster
nodes:
  - role: control-plane
  - role: worker
  - role: worker2
```

4.2 Kubernetes Manifest Highlights

- **ConfigMap:** Supplies **FLASK_ENV=production** and **PORT=5000**.
- **Deployment:** Maintains two replicas with non-root security settings, resource requests/limits, and HTTP readiness/liveness probes.
- **Service:** Uses NodePort, exposing application port 5000 through NodePort 30080.

5. Execution Verification & Evidence Analysis

5.1 Stage 1 — Baseline Application Verification

The standalone Flask application was tested on host port 5000. The supplied evidence records the expected response and HTTP 200 status.

```
URL:      http://127.0.0.1:5000
Output: Hello, Flask!
Status: HTTP 200 — Verified
```

5.2 Stage 2 — Standalone Docker Container Verification

The image **msc-flask-app:1.0.0** was started as **test_flask_app**. The curl response returned HTTP 200 and the expected application payload. The recorded docker ps output reported the container as healthy.

```
docker run -d -p 5000:5000 --name test_flask_app msc-flask-app:1.0.0

curl.exe -i http://127.0.0.1:5000/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.11.16
Content-Type: text/html; charset=utf-8
Content-Length: 13

Hello, Flask!

docker ps:
msc-flask-app:1.0.0 "python run.py"
Up 43 seconds (healthy)
0.0.0.0:5000->5000/tcp
```

5.3 Stage 3 — Kubernetes Workload & Non-Root Verification

After deployment to **msc-cluster**, two replicas were shown in the supplied evidence. They were scheduled on the two worker nodes, demonstrating distribution of the workload across the cluster.

```
flask-app-7f946497b7-nmjbs 1/1 Running 0 ... msc-cluster-worker
flask-app-7f946497b7-p5gzn 1/1 Running 0 ... msc-cluster-worker2

uid=10001(appuser) gid=10001(appgroup) groups=10001(appgroup)
```

The identity check confirms that the running application process is using the dedicated non-root UID/GID rather than root.

5.4 Stage 4 — Kubernetes Service Endpoint Routing

Service-layer traffic was verified by forwarding host port 8080 to the Flask service port 5000. The supplied terminal evidence records a successful HTTP response and the expected application payload.

```
curl.exe -i http://127.0.0.1:8080/

HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.11.16
Content-Type: text/html; charset=utf-8
Content-Length: 13
Connection: close

Hello, Flask!
```

The endpoint test provides an end-to-end verification path from the local forwarded endpoint through the Kubernetes Service to the Flask application.

6. Project Verification & Audit Matrix

Requirement	Implementation	Status	Evidence
Containerization	Python 3.11-slim; isolated dependencies	Passed	Dockerfile; compose.yaml
Non-Root Execution	appuser, UID 10001	Passed	container_uid.txt; container_user.txt
Healthcheck	urllib.request probe	Passed	healthcheck.txt
Vulnerability Audit	Docker Scout + SBOM	Passed	security/sbom.json; docker_scout_vulns.txt
Registry Push	Docker Hub tagged image	Passed	harineshmk/msc-de1-flask-app:1.0.0
Multi-Node Cluster	1 control-plane + 2 workers	Passed	cluster-config.yaml
Declarative Orchestration	ConfigMap + Deployment + Service	Passed	k8s/*.yaml
Pod Scheduling	Replicas on both workers	Passed	k8s_pods.txt
Service Verification	Endpoint routing verified	Passed	k8s_endpoint_response.txt
Clean Repo/Teardown	Executables removed; repo synchronized	Passed	GitHub repository

7. Conclusion

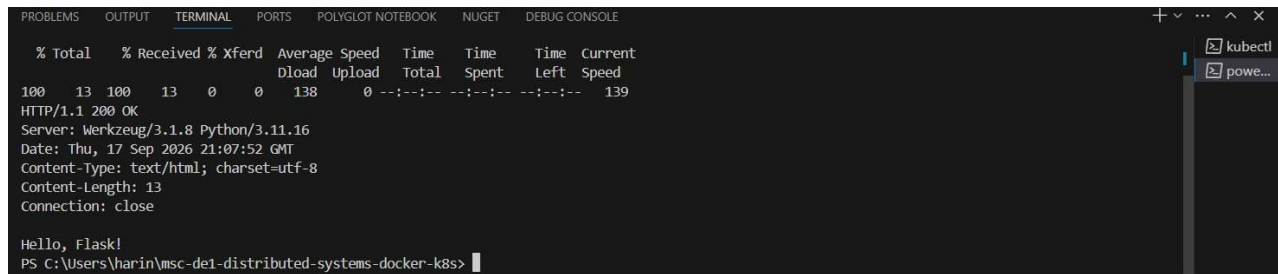
The supplied project material documents a complete workflow from the Flask application through Docker containerization and Kubernetes orchestration. The application is packaged using Python 3.11-slim, configured for non-root execution, equipped with a health check, and supported by documented security-audit artifacts.

The Kubernetes layer defines a three-node Kind cluster with one control-plane node and two workers. A Deployment maintains two replicas, and the supplied runtime evidence shows those replicas running on the two worker nodes. The UID/GID check confirms execution as appuser with UID/GID 10001.

The service-routing test returns HTTP 200 and the expected **Hello, Flask!** payload through the forwarded Kubernetes endpoint. Together, the runtime and audit evidence provide a traceable record of container health, security posture, scheduling, and network service verification.

Appendix A — Kubernetes Endpoint Evidence

The supplied screenshot below records the terminal output for the Kubernetes endpoint test, including the successful HTTP 200 response, server information, content length, and returned Flask payload.



```
PROBLEMS OUTPUT TERMINAL PORTS POLYGLOT NOTEBOOK NUGET DEBUG CONSOLE
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 13 100 13 0 0 138 0 --:--:-- --:--:-- --:--:-- 139
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.11.16
Date: Thu, 17 Sep 2026 21:07:52 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Connection: close

Hello, Flask!
PS C:\Users\harin\msc-de1-distributed-systems-docker-k8s>
```

Figure A.1 — Kubernetes endpoint response captured from the supplied terminal evidence.

Appendix B — Key Submission Artifacts

- **Docker:** Dockerfile, compose.yaml, health-check and image-layer evidence.
- **Security:** Docker Scout vulnerability output and SBOM.
- **Kubernetes:** Kind configuration plus ConfigMap, Deployment and Service manifests.
- **Runtime:** pod, service, deployment, endpoint, UID and user evidence.
- **Source control:** GitHub repository containing the project artifacts.